InternViT

论文：《InternVL: Scaling up Vision Foundation Models and Aligning for Generic Visual-Linguistic Tasks》
模型仓库：OpenGVLab/InternVL1.0

InternViT 是 InternVL 系列多模态大模型（InternVL 3、InternVL 3.5）的视觉编码器。

在模型结构上，InternViT 相比早期的 ViT(2020, Google Research)结构，融合了 RMSNorm、QK Norm、LayerScale、DropPath、FlashAttention 等多项现代 Transformer 架构技术。

在模型训练上，InternViT 融合了对比学习任务和生成任务，生成任务参考了BLIP-2(2023, Salesforce Research)的 Qformer 训练模式（参考MLP vs. Q-Former），但是又可以灵活遵循LLaVA-1.5(2023, Microsoft Research)方案对齐。

1. 模型架构

整体是一个 ViT 编码器结构（Vision Transformer Encoder），包括：

- Patch Embedding
- Transformer Encoder
- 输出整合（[cls]、last_hidden_state）

但 InternViT 相比早期的 ViT(2020, Google Research)结构，融合了 RMSNorm、QK Norm、LayerScale、DropPath、FlashAttention 等多项现代 Transformer 架构技术。

1.1 RMSNorm

LLaMA 以来的大模型大多都使用 RMSNorm。Qwen2-VL也在 ViT 上使用 RMSNorm 让 ViT 更接近语言模型架构。

RMSNorm 通过均方根值缩放输入，保留原始均值信息，其公式是：

$$y_i = \frac{x_i}{\mathrm{RMS}(x)} * \gamma_i, \text{ where } \mathrm{RMS}(x) = \sqrt{\epsilon + \frac{1}{n}\sum_{i=1}^{n} x_i^2}$$

其中 RMS 是在最后一个维度 (hidden_dim) 上进行计算的。可训练参数为向量 γ。

```python
class InternRMSNorm(nn.Module):
    def __init__(self, hidden_size, eps=1e-6):
        super().__init__()
        self.weight = nn.Parameter(torch.ones(hidden_size))
        self.variance_epsilon = eps

    def forward(self, hidden_states):
        input_dtype = hidden_states.dtype
        hidden_states = hidden_states.to(torch.float32)
        variance = hidden_states.pow(2).mean(-1, keepdim=True)
        hidden_states = hidden_states * torch.rsqrt(variance + self.variance_epsilon)
        return self.weight * hidden_states.to(input_dtype)
```

1.2 QK Norm

QK Norm 是近期开源的文本大模型使用的技术，目标是为了降低数值波动、提升稳定性（使 Softmax 函数

在不牺牲表达能力的情况下不那么容易出现饱和）。

做法上是在 Q、K 上分别做 L2 Norm 实现把 Q、K 两个 vector（这里是逐 token 计算视角，因此是 vector）转化为单位向量，这样的话 QK 的单点乘积就可以变成 $Q'K'^T = cosine_similarity(Q', K')$

其中 $Q' = Q/\|Q\|$，$K' = K/\|K\|$，而$cosine_similarity$是限制在$[-1, 1]$的，避免出现 Softmax 饱和问题。

```python
class InternAttention(nn.Module):
    """Multi-headed attention from 'Attention Is All You Need' paper"""

    def __init__(self, config: InternVisionConfig):
        self.qkv = nn.Linear(self.embed_dim, 3 * self.embed_dim, bias=config.qkv_bias)
        self.attn_drop = nn.Dropout(config.attention_dropout)
        self.proj_drop = nn.Dropout(config.dropout)

        self.qk_normalization = config.qk_normalization

        if self.qk_normalization:
            self.q_norm = InternRMSNorm(self.embed_dim, eps=config.layer_norm_eps)
            self.k_norm = InternRMSNorm(self.embed_dim, eps=config.layer_norm_eps)

    def _naive_attn(self, x):
        B, N, C = x.shape
        qkv = self.qkv(x).reshape(B, N, 3, self.num_heads, C // self.num_heads).permute(2, 0, 3, 1, 4)
        q, k, v = qkv.unbind(0)  # make torchscript happy (cannot use tensor as tuple)

        if self.qk_normalization:
            B_, H_, N_, D_ = q.shape
            q = self.q_norm(q.transpose(1, 2).flatten(-2, -1)).view(B_, N_, H_, D_).transpose(1, 2)
            k = self.k_norm(k.transpose(1, 2).flatten(-2, -1)).view(B_, N_, H_, D_).transpose(1, 2)

        attn = ((q * self.scale) @ k.transpose(-2, -1))
```

1.3 LayerScale & DropPath

DropPath 和 Dropout 一样也是正则化方法。

传统的 Dropout 是在 **神经元层面**随机丢弃激活值，而 DropPath 是在 **网络路径（整层残差分支）层面** 随机丢弃。也就是说，在每次前向传播中：

- 某些残差连接（Residual Branch）会被随机关闭；
- 相当于“跳过”某一层的 Transformer block；
- 从而让不同样本看到的模型结构略有不同。

而且在 DropPath 中，InternViT 还使用了两组可学习 LayerScale：`self.ls1`和 `self.ls2`：

```python
class InternVisionEncoderLayer(nn.Module):
    def __init__(self, config: InternVisionConfig, drop_path_rate: float):
        super().__init__()
        self.embed_dim = config.hidden_size
        self.intermediate_size = config.intermediate_size

        self.attn = InternAttention(config)
```

```
        self.mlp = InternMLP(config)
        self.norm1 = InternRMSNorm(self.embed_dim, eps=config.layer_norm_eps)
        self.norm2 = InternRMSNorm(self.embed_dim, eps=config.layer_norm_eps)

        self.ls1 = nn.Parameter(config.initializer_factor * torch.ones(self.embed_dim))
        self.ls2 = nn.Parameter(config.initializer_factor * torch.ones(self.embed_dim))
        self.drop_path1 = DropPath(drop_path_rate) if drop_path_rate > 0. else nn.Identity()
        self.drop_path2 = DropPath(drop_path_rate) if drop_path_rate > 0. else nn.Identity()

    def forward(
            self,
            hidden_states: torch.Tensor,
    ) -> Tuple[torch.FloatTensor, Optional[torch.FloatTensor], Optional[Tuple[torch.FloatTensor]]]:
        """
        Args:
            hidden_states (`Tuple[torch.FloatTensor, Optional[torch.FloatTensor]]`): input to the layer of shape `(batch, seq_len, embed_dim)`
        """
        hidden_states = hidden_states + self.drop_path1(self.attn(self.norm1(hidden_states)) * self.ls1)

        hidden_states = hidden_states + self.drop_path2(self.mlp(self.norm2(hidden_states)) * self.ls2)

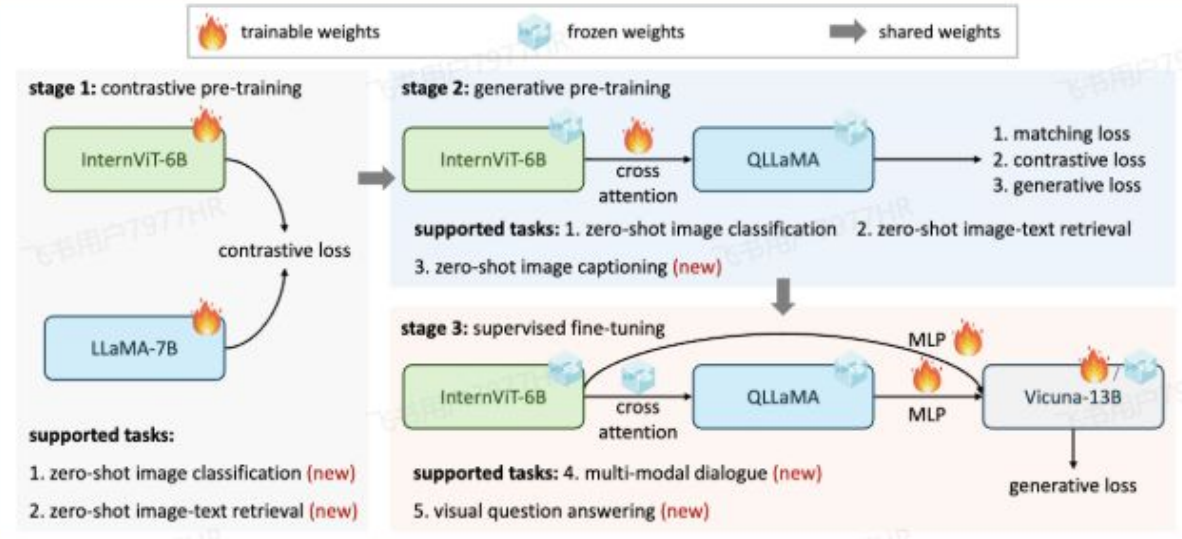
        return hidden_states
```

值得注意的是，InternViT 使用的位置编码和 ViT(2020, Google Research)一样仍然是 **可学习的 (learnable) 绝对位置编码**，参考ViT 的位置编码：

```
self.position_embedding = nn.Parameter(
    torch.randn(1, self.num_positions, self.embed_dim)
)
```

2. 训练

InternViT 的训练通过 QLLaMA 中间件对齐视觉与语言特征空间，从对比学习到生成学习的策略演进，有效利用不同质量数据。三个阶段分别采用196/224/364像素分辨率。



Stage 1：视觉-语言对比训练

1. **模型初始化**：图像编码器 InternViT-6B 采用 BEiT 初始化方法随机初始化，文本编码器 LLaMA-7B 加载多语言预训练权重，全部参数可训练。
2. **数据与优化**：使用 4.98B 过滤后的图像-文本对（来源包括 LAION-en, LAION-multi, LAION-COCO, COYO, Wukong 等），采用 AdamW 优化器（β1=0.9, β2=0.95），图像/文本编码器初始学习率分别为 1e-3/1e-4，cosine 衰减 LR。
3. 初始化 196×196 分辨率训练时掩码 50% 图像 token。最后 0.5B 训练数据过渡到 224×224 分辨率阶段，取消掩码。

Stage 2：视觉-语言生成训练

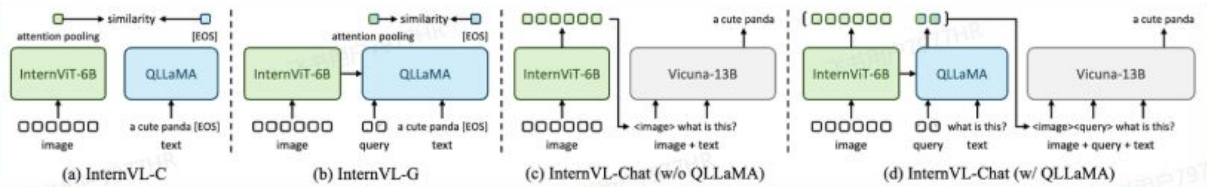
这里引入了一个 QLLaMA 权重，这是一个经过视觉语言对齐的 LLaMA 权重。

1. **参数冻结**：InternViT-6B 和 QLLaMA 冻结权重，仅训练新增的 96 个可学习查询和交叉注意力层（本质就是 Q-Former 结构，1B 参数）。
2. **数据升级**：使用严格过滤后的 1.03B 高质量图文对，包含 ITC/ITM/ITG 综合损失，参考BLIP-2(2023, Salesforce Research)。
3. AdamW 优化器（β1=0.9, β2=0.98），峰值学习率 5e-5，20K batchsize。

Stage 3：监督微调

1. **两种配置**：
   - (a) 独立使用 InternViT-6B：遵循LLaVA-1.5(2023, Microsoft Research)方案，分两阶段微调 MLP 层和 LLM。
   - (b) 全模型整合：结合 InternViT-6B 和 QLLaMA（本身可以看作一个大型 Q-Former），扩展数据集至 4M 样本，批量增至 512。
2. **训练策略**：
   - 使用混合指令数据集（COCO Caption、VQAv2 等）
   - （可选）冻结 LLM 解码器训练，保留原始语言能力。

这里读者可能有点疑惑这个 InternViT-6B+QLLaMA 如何使用，这个看下面这个使用场景图就知道，(c) 就是直接使用 InternViT-6B、(d) 是使用 InternViT-6B+QLLaMA：



3. 实验

论文里关注了 classification、image-text retrieval 这样的 ViT 传统 benchmark，我们主要关注多模态理解生成的表现：

method	visual encoder	glue layer	LLM	Res.	PT	SFT	train. param	image captioning			visual question answering				dialogue	
								COCO	Flickr	NoCaps	VQA^{v2}	GQA	VizWiz	VQA^{T}	MME	POPE
InstructBLIP [34]	EVA-g	QFormer	Vicuna-7B	224	129M	1.2M	188M	–	82.4	123.1	–	49.2	34.5	50.1	–	–
BLIP-2 [81]	EVA-g	QFormer	Vicuna-13B	224	129M	–	188M	–	71.6	103.9	41.0	41.0	19.6	42.5	1293.8	85.3
InstructBLIP [34]	EVA-g	QFormer	Vicuna-13B	224	129M	1.2M	188M	–	82.8	121.9	–	49.5	33.4	50.7	1212.8	78.9
InternVL-Chat (ours)	IViT-6B	QLLaMA	Vicuna-7B	224	1.0B	4.0M	64M	141.4*	89.7	120.5	72.3*	57.7*	44.5	42.1	1298.5	85.2
InternVL-Chat (ours)	IViT-6B	QLLaMA	Vicuna-13B	224	1.0B	4.0M	90M	142.4*	89.9	123.1	71.7*	59.5*	54.0	49.1	1317.2	85.4
Shikra [21]	CLIP-L	Linear	Vicuna-13B	224	600K	5.5M	7B	117.5*	73.9	–	77.4*	–	–	–	–	–
IDEFICS-80B [66]	CLIP-H	Cross-Attn	LLaMA-65B	224	1.6B	–	15B	91.8*	53.7	65.0	60.0	45.2	36.0	30.9	–	–
IDEFICS-80B-I [66]	CLIP-H	Cross-Attn	LLaMA-65B	224	353M	6.7M	15B	117.2*	65.3	104.5	37.4	–	26.0	–	–	–
Qwen-VL [5]	CLIP-G	VL-Adapter	Qwen-7B	448	1.4B†	50M†	9.6B	–	85.8	121.4	78.8*	59.3*	35.2	63.8	–	–
Qwen-VL-Chat [5]	CLIP-G	VL-Adapter	Qwen-7B	448	1.4B†	50M†	9.6B	–	81.0	120.2	78.2*	57.5*	38.9	**61.5**	1487.5	–
LLaVA-1.5 [91]	$CLIP\text{-}L_{336}$	MLP	Vicuna-7B	336	558K	665K	7B	–	–	–	78.5*	62.0*	50.0	58.2	1510.7	85.9
LLaVA-1.5 [91]	$CLIP\text{-}L_{336}$	MLP	Vicuna-13B	336	558K	665K	13B	–	–	–	80.0*	63.3*	53.6	61.3	1531.3	85.9
InternVL-Chat (ours)	IViT-6B	MLP	Vicuna-7B	336	558K	665K	7B	–	–	–	79.3*	62.9*	52.5	57.0	1525.1	86.4
InternVL-Chat (ours)	IViT-6B	MLP	Vicuna-13B	336	558K	665K	13B	–	–	–	80.2*	63.9*	54.6	58.7	1546.9	87.1
InternVL-Chat (ours)	IViT-6B	QLLaMA	Vicuna-13B	336	1.0B	4.0M	13B	**146.2***	**92.2**	**126.2**	**81.2***	**66.6***	**58.5**	**61.5**	**1586.4**	**87.6**